

Introduction to Node.js

ผศ. ดร. เกี๊จแกว ฐเนศวร

kejkaew.tha@mail.kmutt.ac.th

ตารางเรียน

สัปดาห์	วันที่	หัวข้อ/รายละเอียด	Assignment (ผู้ตรวจ)	ส่ง Assignment
1	4/8/2569	Introduction to Web Development & HTML5	Assignment 1 (KT,KN)	1
2	11/8/2569	CSS3 Fundamentals	Assignment 2 (KT, KN)	1,2
3	18/8/2569	Advanced CSS & UI Layouts	Assignment 3 (KT)	1,2,3
4	25/8/2569	Responsive Web Design	Assignment 4 (KT, KN)	2,3,4
5	1/9/2569	JavaScript 1: Variables, Data Types, DOM & Event Handling	Assignment 5 (KT, KN)	3,4,5
6	8/9/2569	JavaScript 2: Arrays, Objects & Logic Control	Assignment 6 (KT)	4,5,6
7	15/9/2569	JavaScript 3: DOM Manipulation	Assignment 7 (KT, KN)	5,6,7
8	22/9/2569	Asynchronous JS, AJAX & JSON Data Flow	Assignment 8 (KT, KN)	6,7,8
9	29/9/2569	Introduction to Node.js	Assignment 9 (KT)	7,8,9
10	6/10/2569	Express Basics & RESTful API (CRUD)	Assignment 10 (KT, KN)	8,9,10
	13/10/2569	หยุด		
	20/10/2569	หยุด		
	27/10/2569	หยุด		
11	3/11/2569	Microservices Architecture Concept & Hands-on	Assignment 11 (KT, KN)	9,10,11
12	10/11/2569	Database Integration (MySQL / Firebase)	Assignment 12 (KT, KN)	10,11,12
13	17/11/2569			11,12
14	24/11/2569	ส่งโปรเจก (9:00-16:00)	KT	12
15	12/2569	สอบปลายภาค ในตาราง		

สัปดาห์ที่แล้ว

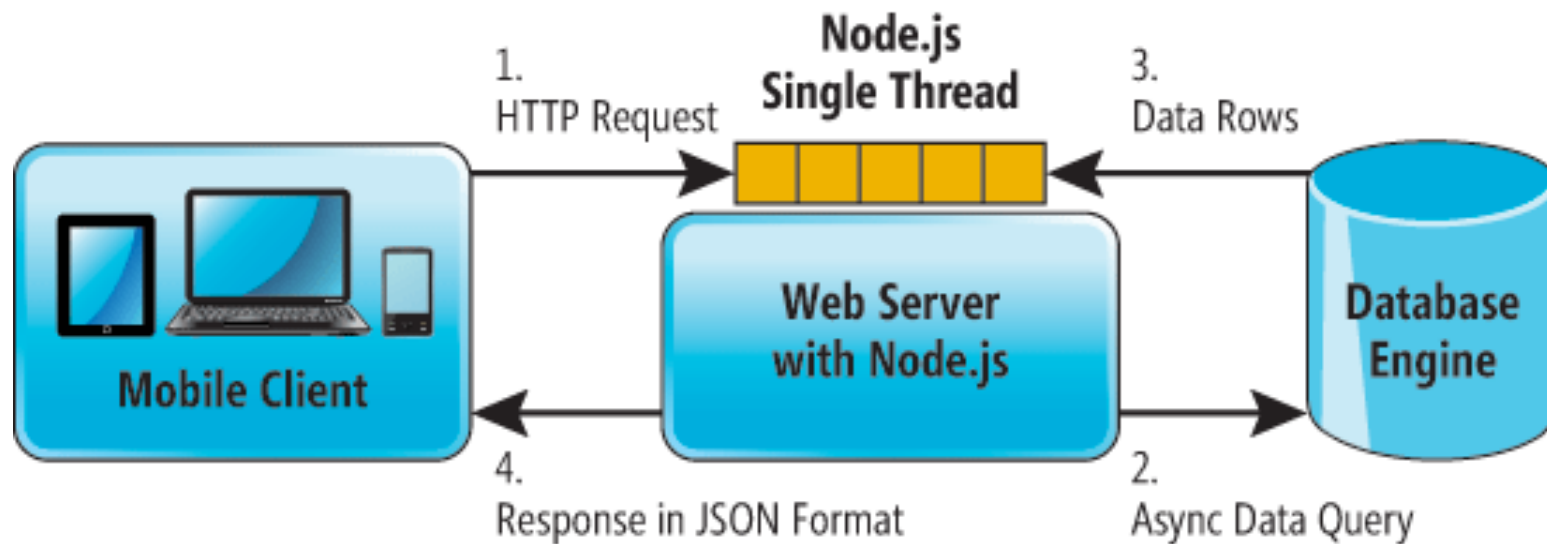
- AJAX : Asynchronous JavaScript And XML
- JSON: JavaScript Object Notation.
- Assignment 8

เรื่องที่จะเรียนวันนี้

- Server-side script
 - Node.js
 - http module
 - File system module
 - Promises
- Assignment 9

บททวน

- Client – Server relationship



What is Node.js?

- Node.js คือเฟรมเวิร์กสภาพแวดล้อมรันไทม์ของ JavaScript (JavaScript Runtime) ที่ทำงานแบบอะซิงโครนัส (Asynchronous) และขับเคลื่อนด้วยเหตุการณ์ (Event-driven)
- ถูกออกแบบมาเพื่อสร้างแอปพลิเคชันบนเครือข่ายที่สามารถรองรับการขยายตัวและรองรับผู้ใช้งานจำนวนมากได้เป็นอย่างดี (Scalable Network Applications)
- Node.js ถูกพัฒนาขึ้นในปี ค.ศ. 2009 (โดย *Ryan Dahl*)
- Node.js เป็นซอฟต์แวร์ฟรี (ไม่มีค่าใช้จ่าย และเป็น Open-source)
- Node.js สามารถทำงานได้บนหลากหลายแพลตฟอร์มหรือระบบปฏิบัติการ (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js นำภาษา JavaScript มาใช้ประมวลผลบนฝั่งเซิร์ฟเวอร์ (Server-side)

Why Node.js?

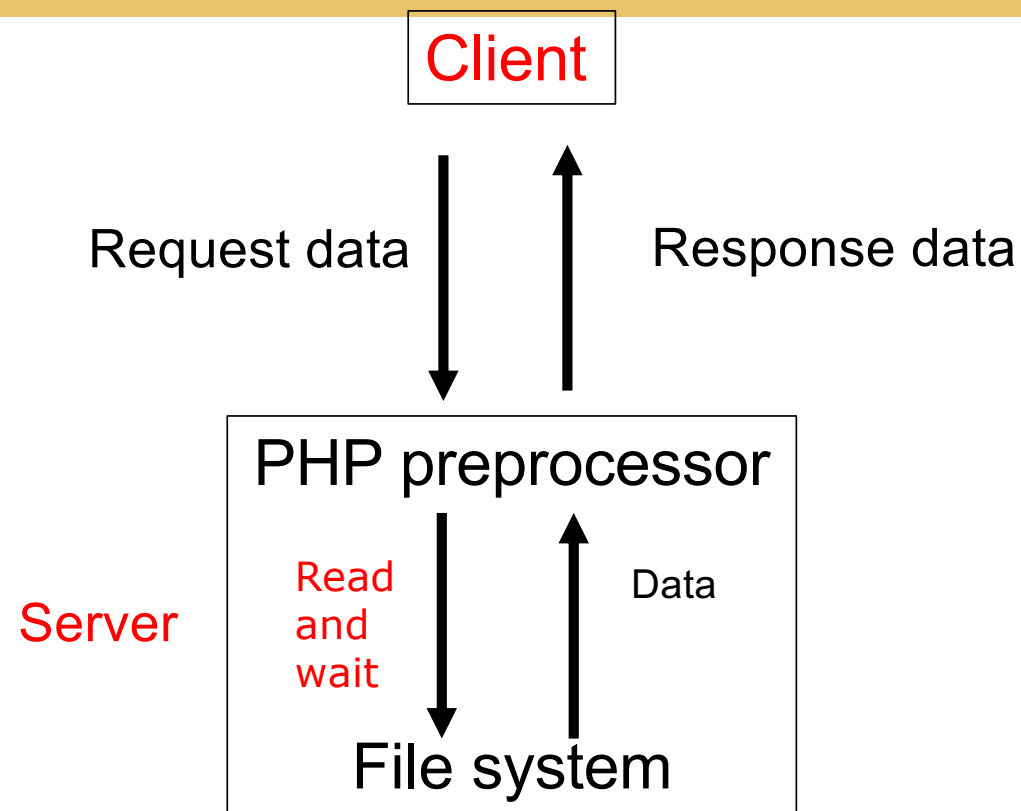
Node.js มีการใช้ asynchronous programming ดังนี้

- Node.js ช่วยตัดปัญหาเรื่องการต้องหยุดรอคอย (Eliminates the waiting) และข้ามไปทำงานกับคำขอ (Request) ถัดไปได้ทันที
- Node.js ทำงานบนเธรดเดียว (Single-threaded) โดยไม่มีการบล็อกการทำงาน (Non-blocking) ด้วยการเขียนโปรแกรมแบบอะซิงโครนัส (Asynchronously programming) ซึ่งช่วยให้ประหยัดการใช้หน่วยความจำ (Memory) ได้อย่างมีประสิทธิภาพสูงมาก

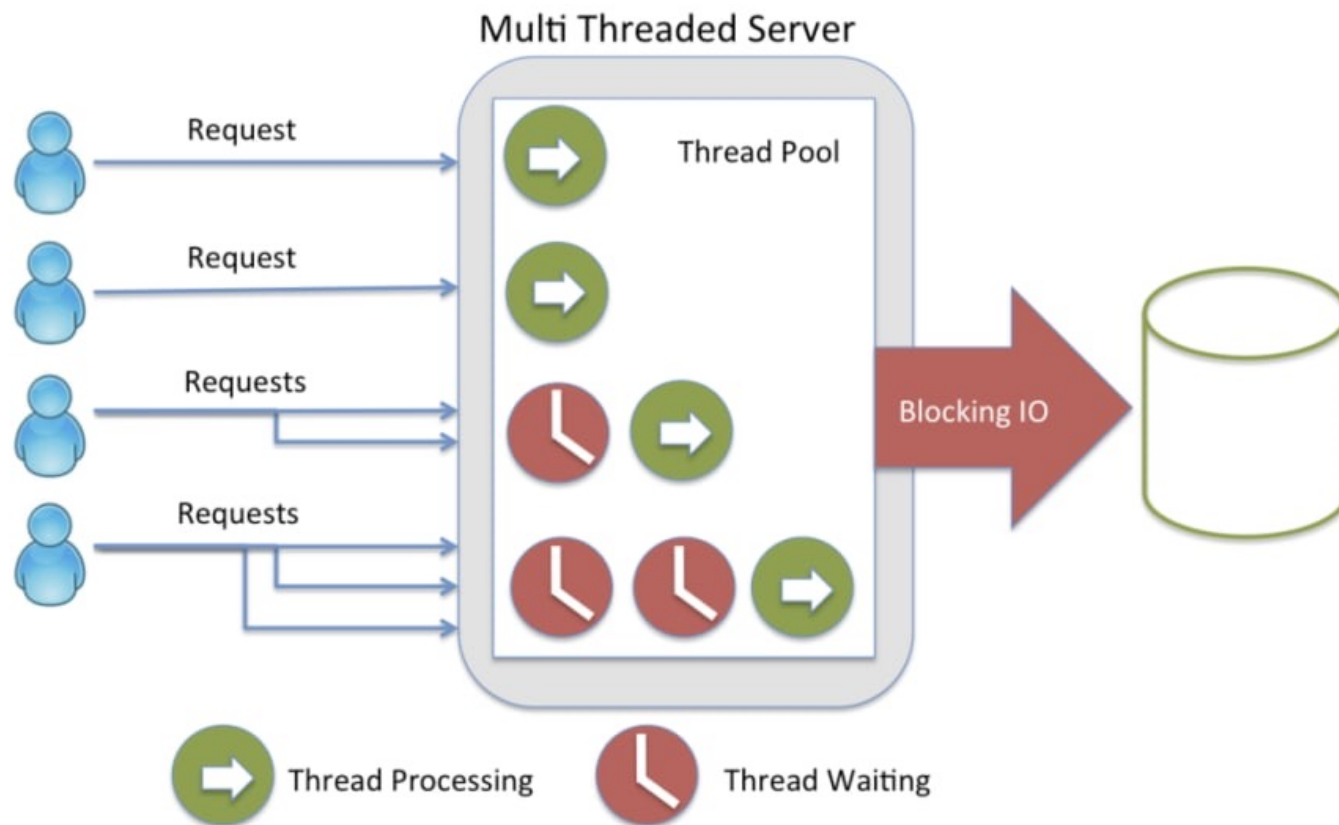
What is Node.js files

- Node.js files จะประกอบไปด้วยชุดคำสั่งทั้งหมดสำหรับเหตุการณ์ (event) บางอย่าง
- เหตุการณ์ที่ว่าจะเกี่ยวข้องกับงานที่ตัว web application ต้องทำบน server เช่น อ่าน อัปเดต เพิ่ม ข้อมูลใน ฐานข้อมูล
- ดังนั้น Node.js files ต้องรันรอเหตุการณ์ที่จะเกิดขึ้น ใน server เสมอ
- Node.js files have extension ".js"

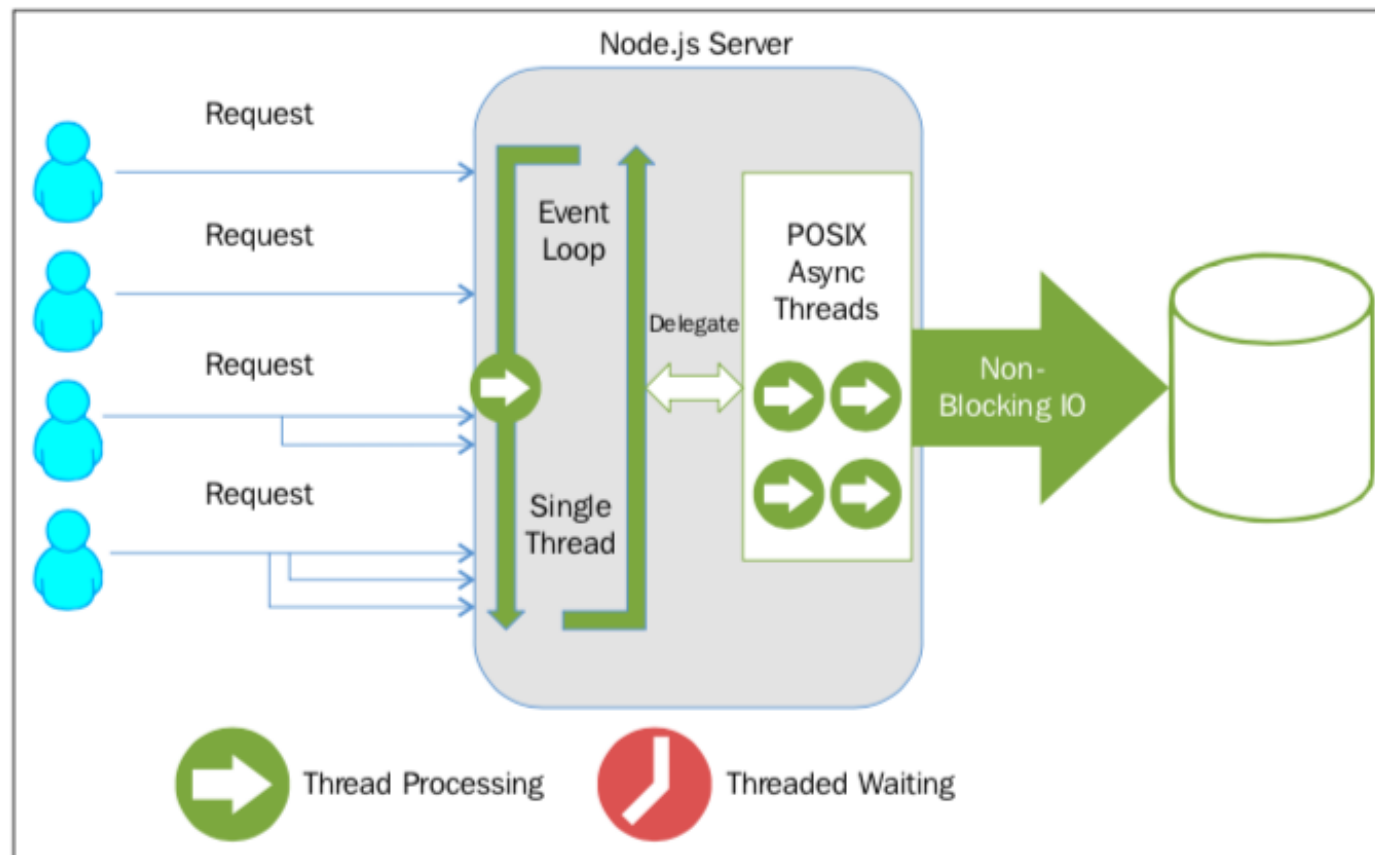
Here is how PHP or ASP handles a file request:



Multi Threaded Server



How Node.js works



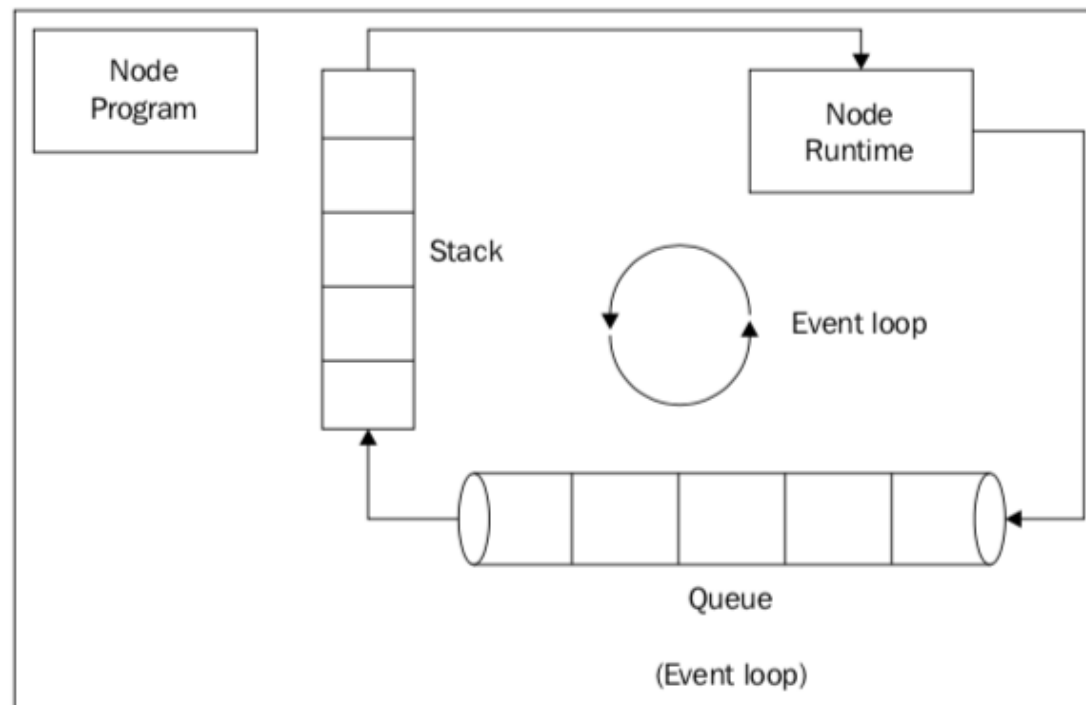
สิ่งที่ควรรู้ใน Node.js

- Callback function = เมื่อทำ task บางอย่างเสร็จแล้ว เช่น กดปุ่ม จะให้ทำ function task อะไรต่อไป
- Blocking or wait = รันแบบ synchronously
- Non-blocking = รันแบบ asynchronously
- Event-driven programming = ลักษณะ flow ของ program ถูกกำหนดโดย user actions เช่น clicks, press

How Event Loop Works

- เนื่องจาก Node.js มีการรัน แบบ asynchronous ดังนั้น เราต้องเข้าใจการทำงานของมันเป็น
- ใน event loop, Node.js มีการจัด queue เพื่อเก็บตัว callback function และ จะทำการ execute เมื่อ stack ว่าง
- ดังนั้นสิ่งที่ เป็น callback function จะรันทีหลังตัว task ปกติ ดังรูป

How Event Loop Works



ตัวอย่าง callback เช่น

```
JS server.js > ...
1 console.log('i am first');
2
3 setTimeout(() => {
4   console.log('i am second');
5 }, 0);
6
7 console.log('i am third');
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
i am first
i am third
i am second
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$
```

Arrow Function

- Arrow functions (ฟังก์ชันลูกศร `() => {}`) ช่วยให้เราสามารถเขียนไวยากรณ์ของฟังก์ชันได้สั้นและกระชับขึ้น
- ตัวอย่างเช่น

```
10 function hello(){  
11     console.log("Hello");  
12 }  
13 hello()
```

```
15 hello = () => {  
16     console.log("Hello");  
17 }  
18 hello()
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js  
Hello
```

Arrow function with parameters

```
20 hello = (message) => {  
21   |   console.log("Hello "+message);  
22 }  
23 hello("Kejkaew")
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js  
Hello Kejkaew
```

Create server

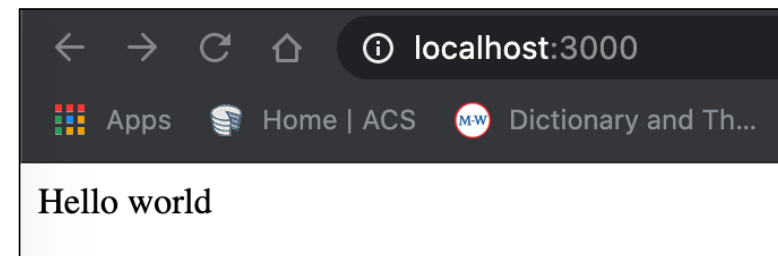
- ลองรัน server

```
25 const http = require('http');
26 const hostname = 'localhost';
27 const port = 3000;
28 const server = http.createServer((req, res) => {
29   res.writeHead(200, {'Content-Type': 'text/html'});
30   res.write("Hello world");
31   res.end();
32 });
33
34 server.listen(port, hostname, () => {
35   console.log(`Server running at http://${hostname}:${port}/`);
36 });
```

Run node

```
kejkaew@Kejkaews-MacBook-Pro-2 node-1 % node server.js
Server running at http://localhost:3000/
```

Output



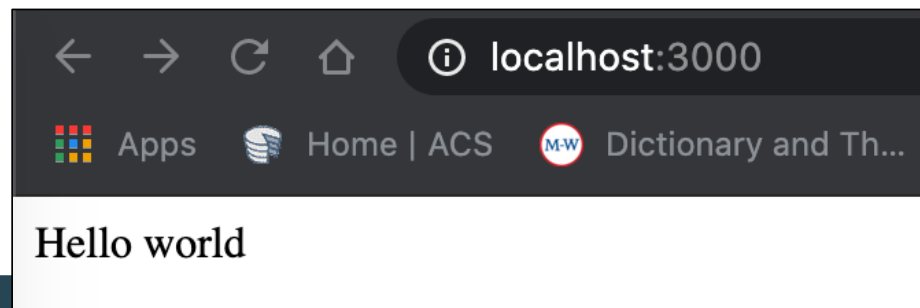
The Built-in HTTP Module

- Node.js มี built-in module ที่ชื่อว่า HTTP ซึ่งใช้สำหรับส่งข้อมูลผ่าน HTTP protocol
- สำหรับการใช้ module นี้ ต้องทำการ import ก่อนโดยใช้ `require()` method:
- HTTP module จะทำการสร้าง HTTP server ที่รอการร้องขอจาก client และส่งข้อมูลกลับไปหา client ตามที่ขอ
- ใช้ `createServer()` method สำหรับ สร้าง HTTP server

ตัวอย่าง

```
25 const http = require('http');
26 const hostname = 'localhost';
27 const port = 3000;
28 const server = http.createServer((req, res) => {
29   res.writeHead(200, {'Content-Type': 'text/html'});
30   res.write("Hello world");
31   res.end();
32 });
33
34 server.listen(port, hostname, () => {
35   console.log(`Server running at http://\${hostname}:\${port}/`);
36 });
```

Output



-
- Function ที่อยู่ใน `http.createServer()` method จะทำงานรอจนกว่าจะมีผู้ใช้ติดต่อเข้ามาที่ port ตามที่เรากำหนด เช่น 3000
 - มี parameters 2 ตัว คือ `request (req)` และ `response (res)`
 - `req` argument คือ request จาก client, as an object
 - `res` argument คือ response จาก server, as an object

-
- ถ้าจะแสดงผล response จาก HTTP server ผ่านทาง web browser ในรูปแบบ HTML
 - เราต้องใส่ **HTTP header และ content type**:
 - argument ตัวแรกของ res.writeHead() method คือ status code ซึ่ง 200 = OK
 - argument ตัวที่สอง คือ object ของ response headers.

-
- `res.write(message)` = ส่ง response ไปที่ client ตามที่ header กำหนด
 - `res.end()` = เป็นการบอกว่า server ส่งข้อมูลต้องการส่ง หมดแล้ว

File system in Node.js

- Read
- Write
- Append
- Delete
- Check if files exists

Read a File in Node.js

- ในการ import ตัว File System module สามารถใช้ `require('fs')`
- `fs.readFile(filename,callback)` method ใช้สำหรับอ่านไฟล์ข้อมูลที่อยู่บนเครื่องคอมพิวเตอร์

ตัวอย่าง

```
40 const fs=require('fs');
41 fs.readFile('test.txt', (err, data) => {
42     if (err)
43         throw err;
44
45     console.log("Content : " + data);
46 });
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
Content : Node.js is an open-source, cross-platform, back-end JavaScript runtime environment that runs on the server side and outside a web browser. Node.js lets developers use JavaScript to write command line tools and for server-side web page content before the page is sent to the user's web browser. Consequently, Node.js represents a paradigm shift in web development around a single programming language, rather than different languages for server-side and client-side development.
from https://en.wikipedia.org/wiki/Node.js
```

Write a File in Node.js

- `fs.writeFile(filename, data, callback)` method ใช้สำหรับการเขียน file
- ถ้ามี filename นั้น อยู่แล้วจะทำการเขียนทับ
- ถ้าไม่มี filename นั้น จะทำการสร้างใหม่

ตัวอย่าง

```
56  const fs = require('fs');
57  const content= "this is the content in the file";
58  fs.writeFile('message.txt', content , (err) => {
59      if (err)
60          throw err;
61      console.log('saved!');
62  });
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
saved!
```

Append a File using Node.js

- `fs.appendFile(filename,data,callback)` method เป็นการเขียน data ใหม่เข้าไปใส่ใน filename ที่มีอยู่แล้ว
- ถ้าไม่มี filename นั้น จะทำการสร้างใหม่

ตัวอย่าง

```
66 const fs = require('fs');
67 const new_data = "This data will be appended at the end of the file.";
68 fs.appendFile('message.txt', new_data , (err) => {
69     if(err)
70         throw err;
71     console.log('The new_content was appended successfully');
72 });
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
The new_content was appended successfully
```

Delete a File in Node.js

- ถ้าต้องการลบ filename ให้ใช้ `fs.unlink(pathของ filename, callback)` method.

```
64 const fs = require('fs');
65 const filename = 'message.txt';
66 fs.unlink(filename, (err) => {
67     if (err)
68         throw err;
69     console.log('File deleted successfully');
70 });
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
File deleted successfully
```

Check if files exists

- `fs.access(path ของ filename, mode, callback)`
- เป็น method ที่ใช้สำหรับตรวจสอบว่าการเข้าถึง filename นั้น
- mode argument ใช้เพื่อกำหนดว่าจะเป็นการเข้าถึง filename นั้นแบบไหน
 - F_OK มี file นั้นอยู่หรือไม่
 - R_OK อ่าน file นั้นได้หรือไม่
 - W_OK เขียน file นั้นได้หรือไม่

ตัวอย่าง

```
72  const fs = require('fs');
73  const path = './test.txt';
74  fs.access(path, fs.F_OK, (err) => {
75    if (err) {
76      console.log("File not found");
77      return;
78    }else{
79      console.log("Exist!");
80    }
81  });
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
Exist!
```

Callback hell

```
1 // Callback Hell
2
3
4 a(function (resultsFromA) {
5     b(resultsFromA, function (resultsFromB) {
6         c(resultsFromB, function (resultsFromC) {
7             d(resultsFromC, function (resultsFromD) {
8                 e(resultsFromD, function (resultsFromE) {
9                     f(resultsFromE, function (resultsFromF) {
10                         console.log(resultsFromF);
11                     })
12                 })
13             })
14         })
15     })
16 });
17
```

Nested callbacks stacked below one another forming a pyramid structure

Promises

- Promises เป็นวิธีที่ใช้จัดการกับ **asynchronous code** เพื่อไม่ให้เกิด callback hell
- เมื่อ promise ถูกเรียกใช้ มันจะอยู่ในสถานะ pending state ซึ่งหมายความว่า callback ทุกตัวที่มี promise ยังทำงานอยู่ และเมื่อตัว callback ก่อนหน้าทำงานสำเร็จ (resolve) แล้ว callback ตัวนั้นก็จะเริ่มทำงานต่อ

```
1 // Callback Hell
2
3
4 a(function (resultsFromA) {
5   b(resultsFromA, function (resultsFromB) {
6     c(resultsFromB, function (resultsFromC) {
7       d(resultsFromC, function (resultsFromD) {
8         e(resultsFromD, function (resultsFromE) {
9           f(resultsFromE, function (resultsFromF) {
10             console.log(resultsFromF);
11           })
12         })
13       })
14     })
15   });
16 });
17
```

- ดังนั้นในการสร้าง promise สุดท้ายแล้วจะเกิดได้สองสถานะ คือ resolved state หรือ rejected state ไปเรื่อยๆ จนจบ chain ของ promise
- Promise API เริ่มต้นโดยการเรียกใช้ Promise constructor ซึ่งมี syntax คือ `new Promise(callback):`
- `Promise.then(onFulfilled, onRejected)` สามารถใส่ argument ได้ 2 แบบ คือ callback สำหรับ resolved state (`onFulfilled`) และ callback สำหรับ rejected state (`onRejected`) แต่สามารถใส่แค่ตัวเดียว คือ `onFulfilled` ก็ได้
- `Promise.catch(onRejected)` ใช้สำหรับจัดการ rejected state ได้

วิธีการใช้ promise ที่สร้างไว้

- ในการใช้ promise ถ้าทำงานสำเร็จจะ return resolve method พร้อมกับค่าที่ได้จากการทำงานสำเร็จ นั้นไปทำงานต่อใน then()
- ต่อจาก then() method จะเป็น catch() method ใช้สำหรับจัดการกับ error ที่ได้จาก reject method ใน promise

ตัวอย่าง ที่ไม่ใช่ promises: อ่านและเขียน file ใหม่

```
84  const fs=require('fs');
85  fs.readFile('test.txt', (err, data) => {
86      if (err)
87          throw err;
88      else{
89          fs.writeFile('message.txt', data , (err) => {
90              if (err)
91                  throw err;
92              console.log('saved!');
93          });
94      }
95  });
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
saved!
```

ตัวอย่าง ที่ใช้ promises: อ่านและเขียน file ใหม่

```
114 const fs = require('fs');
115 const readData = () => {
116     return new Promise((resolve, reject) => {
117         fs.readFile('test.txt', (err, data) => {
118             if (err) {
119                 reject(err);
120             } else {
121                 resolve(data);
122             }
123         });
124     });
125 }
```

```
127 const writeData = (data) => {
128     return new Promise((resolve, reject) => {
129         fs.writeFile('message.txt', data, (err) => {
130             if (err) {
131                 reject(err);
132             } else {
133                 resolve("saved!");
134             }
135         });
136     });
137 }
```

Call function

```
139 readData()
140     .then((data) => {
141         return writeData(data);
142     })
143     .then((result) => {
144         console.log(result);
145     })
146     .catch((err) => {
147         console.error(err);
148     });
```

Output

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
saved!
```

การใช้ Async/Await

- ทำให้การใช้ promise ใน JavaScript เข้าใจได้ง่ายขึ้น
- สามารถทำได้โดย
 - ประกาศ async หน้า function ที่จะทำการเรียก function ที่มี promise
 - ประกาศ await หน้า ตัวแปรผลลัพธ์ของการทำงานของ function ที่มี promise หรือ ตัวแปรที่เรารอผลจาก server เช่น response

การใช้ Async/Await ร่วมกับ require('fs/promises')

- เป็นการนำเข้าโมดูล File System ของ Node.js ในเวอร์ชันที่เป็น Promise-based API ซึ่งถูกออกแบบมาให้ทำงานคู่กับ async / await แทนการเขียน Callback ซ้อนกันหลายชั้น (Callback Hell) หรือการเขียน new Promise(...) ด้วยตัวเอง
- ดังนั้น คำสั่ง อ่าน เขียน เพิ่ม ลบ ทุกตัว จะคืนค่าเป็น Promise จึงต้องใส่คำว่า await ไว้หน้าคำสั่ง และครอบด้วยฟังก์ชันที่เป็น async เสมอ

ตัวอย่างเช่น การอ่านและเขียน file ใหม่

```
152 const fs = require('fs/promises');
153 async function readData() {
154     try {
155         const data = await fs.readFile('test.txt');
156         return data;
157     } catch (err) {
158         throw err;
159     }
160 }
```

```
162 async function writeData(data) {
163     try {
164         await fs.writeFile('message.txt', data);
165         return "saved!";
166     } catch (err) {
167         throw err;
168     }
169 }
```

```
171 async function readWriteData() {
172     try {
173         const data = await readData();
174         const result = await writeData(data);
175         console.log(result);
176     } catch (err) {
177         console.error(err);
178     }
179 }
180
181 readWriteData();
```

Call function

Read and Write JSON File

- เหมือนกับที่อ่านและเขียน text file

```
185 const fs = require('fs/promises');
186 const readData = async () => {
187     try {
188         const data = await fs.readFile('jfile.json', 'utf8');
189         console.log(data);
190         return data;
191     } catch (err) {
192         throw err;
193     }
194 }
```

```
196 const writeData = async (data) => {
197     try {
198         await fs.writeFile('new_jfile.json', data);
199         return "saved!";
200     } catch (err) {
201         throw err;
202     }
203 }
```

```
205  const readWriteData = async () => {
206      try {
207          const data = await readData();
208          const result = await writeData(data);
209          console.log(result);
210      } catch (err) {
211          console.error(err);
212      }
213  }
214
215  readWriteData();    Call function
```

```
(base) Kejkaews-MacBook-Pro:eventloop Kejkaew$ node server.js
{
  "name": "Joe",
  "lastname": "Doe",
  "age": 55
}
saved!
```

Output

Assignment 9

- assignment9.zip file
- ให้นักศึกษาทำการแก้ไข json file โดยใช้ node.js และ Promise มีขั้นตอนดังนี้
 - อ่านไฟล์ cloth1.json
 - แก้ไขข้อมูล โดยทำการเพิ่ม จำนวนเสื้อผ้าที่เหลืออยู่ในแต่ละ item ตามข้อมูลที่กำหนดให้
 - ทำการ write ข้อมูลที่เป็น object ที่แก้ไขแล้วลงใน file ใหม่ที่ชื่อว่า new_cloth.json และแสดงผลบน browser ตามตัวอย่าง
 - ขั้นตอนทั้งหมดจะทำงาน ก็ต่อเมื่อ เปิด browser
- ส่งด้วยนะคะ

ผลที่แสดงบน browser

```
[
  {
    "brandname": "A.K.O.O Clothing",
    "price": "200",
    "pic": "top1.jpg",
    "stock": 12
  },
  {
    "brandname": "B.K.T Clothing",
    "price": "500",
    "pic": "top2.jpg",
    "stock": 13
  },
  {
    "brandname": "R.O.V. Clothing",
    "price": "700",
    "pic": "top3.jpg",
    "stock": 50
  },
  {
    "brandname": "C.O.M Clothing",
    "price": "800",
    "pic": "top4.jpg",
    "stock": 22
  },
  {
    "brandname": "W.W.E Clothing",
    "price": "100",
    "pic": "top5.jpg",
    "stock": 55
  },
  {
    "brandname": "C.E.N Clothing",
    "price": "600",
    "pic": "top6.jpg",
    "stock": 87
  },
]
```

PopQuiz in LEB2
